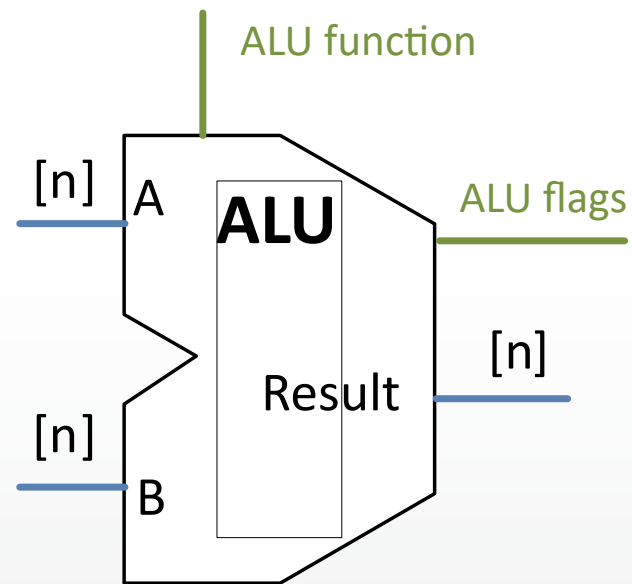


9. Synthesisable SystemVerilog

- Combinational logic

9.1: ALU

ALU module



NB RISC-V does not have arithmetic ALU flags.

Approaches to ALU synthesis

- Methods to synthesise ADD, SUB
 - 1) use $a+b$ in 8-bits , but add extra logic (or instructions) to extract carry
 - 2) synthesise ALU on structural level using 1-bit slices.
- Subtraction can be implemented using 2's complement subtraction rule:
 - $A - B = A + \sim B + 1$
 - This way the same hardware can be used for ADD and SUB
- 2's complement overflow (if needed!) can be obtained from simple combinational logic.
- However, we will look at hardware in more detail later

Packages

- Declare opcodes as an enumerated type
- Declare in a package to use in multiple modules

```
package ibex_pkg;

typedef enum logic [6:0] {
    OPCODE_LOAD      = 7'h03,
    OPCODE_MISC_MEM  = 7'h0f,
    OPCODE_OP_IMM     = 7'h13,
    OPCODE_AUIPC      = 7'h17,
    OPCODE_STORE      = 7'h23,
    OPCODE_OP         = 7'h33,
    OPCODE_LUI        = 7'h37,
    OPCODE_BRANCH     = 7'h63,
    OPCODE_JALR       = 7'h67,
    OPCODE_JAL        = 7'h6f,
    OPCODE_SYSTEM     = 7'h73
} opcode_e;
endpackage
```

enumerated type with values

Include vs Package

- ``include` a file of definitions
- ``define` is simply a text substitution

```
`define OP CODE_LOAD 7'h03
`define OP CODE_MISC_MEM 7'h0f
`define OP CODE_OP_IMM 7'h13
`define OP CODE_AUIPC 7'h17
...
```

- A package is compiled and checked separately
- Allows a change without recompiling everything else

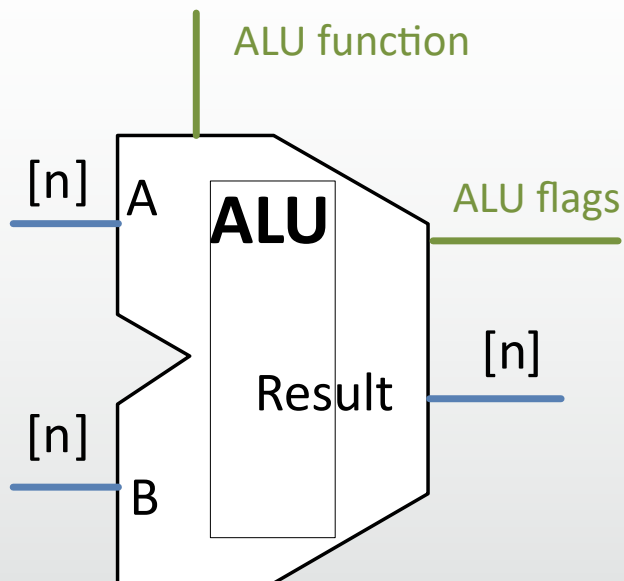
ALU module

```
module alu (  
    input  ibex_pkg::alu_op_e operator_i,  
    input  logic [31:0]      operand_a_i,  
    input  logic [31:0]      operand_b_i;  
    output logic [31:0]      result_o,  
    output logic              comparison_result_o,  
    ...  
);
```

Note ibex coding convention
input _i, output _o

```
typedef enum logic [6:0] {  
    // Arithmetics  
    ALU_ADD,  
    ALU_SUB,  
    // Logics  
    ALU_XOR,  
    ALU_OR,  
    ALU_AND,  
    // Shifts  
    ALU_SRA,  
    ALU_SRL,  
    ALU_SLL,  
    ...  
} alu_op_e;
```

enumerated type - no values



ALU module - cont

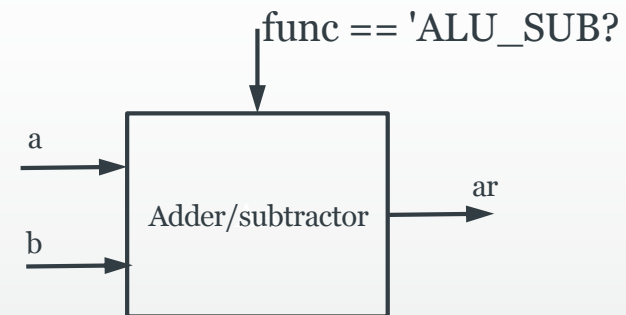
```
// create an n-bit adder
// and then build the ALU around the adder

if(operator_i==ALU_SUB)
    adder_op_b_negate = 1'b1;
else
    adder_op_b_negate = 1'b0;

assign operand_b_neg = ~operand_b_i + 1'b1;
```

Create an adder and logic for
-B needed in subtraction.

The adder result (ar) will be
used in the always_comb
block that follows.



Reverse case

```
unique case (1'b1)
  multdiv_sel_i:      adder_in_b = multdiv_operand_b_i;
  adder_op_b_negate: adder_in_b = operand_b_neg;
  default:            adder_in_b = {operand_b_i, 1'b0};
endcase
```

- Multiplexer
 - adder_in_b is assigned depending on whether multdiv_sel_i *or* adder_op_b_negate is true
 - only one may be true (unique); if neither is true, default is chosen

Adder

```
assign adder_result = adder_in_a + adder_in_b;
```

- This isn't exactly how the Ibex core implements the adder, but it's the same principle.

9.2: Decoder and Program Memory

```
unique case ({instr_alu[31:25], instr_alu[14:12]})
  // RV32I ALU operations
  {7'b000_0000, 3'b000}: alu_operator_o = ALU_ADD;    // Add
  {7'b010_0000, 3'b000}: alu_operator_o = ALU_SUB;    // Sub
  {7'b000_0000, 3'b010}: alu_operator_o = ALU_SLT;    // Set Lower Than
  {7'b000_0000, 3'b011}: alu_operator_o = ALU_SLTU;   // Set Lower Than Unsigned
  {7'b000_0000, 3'b100}: alu_operator_o = ALU_XOR;    // Xor
  {7'b000_0000, 3'b110}: alu_operator_o = ALU_OR;     // Or
  {7'b000_0000, 3'b111}: alu_operator_o = ALU_AND;    // And
  {7'b000_0000, 3'b001}: alu_operator_o = ALU_SLL;    // Shift Left Logical
  {7'b000_0000, 3'b101}: alu_operator_o = ALU_SRL;    // Shift Right Logical
  {7'b010_0000, 3'b101}: alu_operator_o = ALU_SRA;    // Shift Right Arithmetic
```

- R-type instruction (OPCODE_OP = 7'h33)
 - concatenates the funct7 and funct3 fields
 - alu_operator_o is of type alu_op_e (see earlier)
 - bit pattern is undefined (enumerated type)

Program Memory

- The program memory is read-only – ROM.
- Hence, it can be considered as combinational logic.
- Need to initialise
 - Read contents from a file

```
initial  
    $readmemh ("prog.hex", progMem) ;
```

- This works for both simulation and synthesis

Program Memory

```
parameter Psize =32, Isize = 32;
// psize - address width, Isize - instruction width

// program memory declaration, note: 1<<n is same as 2^n
logic [Isize-1:0] progMem[ (1<<Psize)];

// get memory contents from file
initial
    $readmemh("prog.hex", progMem);

// program memory read
always_comb
    I = progMem[address];
```

We don't care about the order of entries in the unpacked array,
so we can simply write the array size

Program file – prog.hex

```
68 65 6C 6C 6F 20 77 6F 72 6C 64 21 00 00 00 00  
55 6E 69 6D 70 6C 65 6D 65 6E 74 65 64 20 73 79  
73 74 65 6D 20 63 61 6C 6C 20 63 61 6C 6C 65 64  
21 0A 00 69 6C 6C 65 67 61 6C 20 69 6E 73 74 72  
75 63 74 69 6F 6E 20 65 78 63 65 70 74 69 6F 6E  
20 68 61 6E 64 6C 65 72 20 65 6E 74 65 72 65 64  
0A 00 65 63 61 6C 6C 20 65 78 63 65 70 74 69 6F  
6E 20 68 61 6E 64 6C 65 72 20 65 6E 74 65 72 65
```

(This is just an example cut from a .hex file.
It's not intended to be meaningful.)